

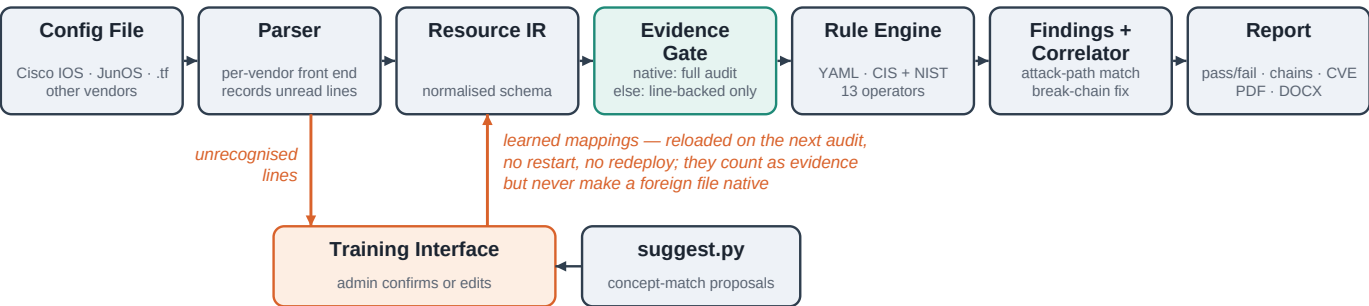
Compliance Auditor — Architecture Document

SIH26155 · AI-Driven Multi-Vendor Network Security Compliance Auditor · Prepared for NTRO

1. Problem

Real network and cloud estates mix vendors: Cisco IOS routers, Juniper JunOS switches and Terraform-defined AWS infrastructure, each with its own configuration syntax. Security benchmarks such as CIS evolve independently of any vendor's format, and an organisation needs one audit process against one baseline rather than a separate checker per vendor. The harder problem is not auditing what a rule set already recognises. It is what happens when the tool meets syntax it has never seen, and how to avoid reporting a verdict on configuration it could not read.

2. System Architecture



The feedback loop from the Training Interface back into the Resource IR is the point of the diagram: it is the difference between this system and a static checker. Cisco IOS parses through `ciscoconfparse2` and JunOS through its own front end, and both share one rule file (`cisco_rules.yaml`); Terraform/AWS parses through `python-hcl2` against `aws_rules.yaml`. Every front end emits the same schema, so the rule engine, correlator and report layer are vendor-blind. A configuration from a vendor with no parser (demonstrated with MikroTik RouterOS and Aruba AOS-CX) runs through the Cisco IOS front end, and the evidence gate ensures it is judged only on lines that were actually read.

3. Data Contracts

Every parser emits a Resource IR and nothing else; every rule is a YAML document, not code. Contracts change only by adding fields, so existing consumers keep working.

Contract	Fields
report	attack_paths, compliance_score (empty when partial or not assessed), cve_context, device, findings, passed, score_breakdown, source, unparsed
finding	cis_control control_ref, explanation, raw_ref (line, snippet), remediation_template, resource_id, rule_id, severity, title
attack path	break_chain, chain_id, contributing_findings, name, narrative, severity
score_breakdown	controls_total, failed_weight, formula, frameworks, lines_taught, not_assessable, partial, recognised_share, rules_evaluated, rules_failed, rules_passed, severity_weights, total_weight

The consequence is demonstrable: adding a vendor means adding a parser, and adding a framework means adding YAML rules tagged with that framework. NIST 800-53 was added exactly this way, with no change to the rule engine, correlator or report layer.

4. The Training Loop

A parser records every line it cannot interpret, with its line number. In the training interface, `suggest.py` proposes a mapping by matching security concepts (Telnet, syslog, SNMP, NTP) rather than vendor syntax: the pattern-matching step the problem statement asks for. An administrator confirms or edits the proposal. The mapping is stored as a normalised prefix of the line's first tokens, so one confirmed mapping applies to any line that starts the same way, and parsers reload mappings on every audit: no restart, no redeployment.

Example (shown in the demo video). A MikroTik export is first reported as **Not assessed**: none of its 16 lines is recognised. An administrator confirms three proposals: `/ip service set telnet disabled=no`, `/ip service set www disabled=no` and `/snmp set enabled=yes`. The re-audit then evaluates 3 of 32 controls and reports one critical and two high findings, each pointing at the line that was taught.

Safeguards. Taught values are typed before any rule sees them. List attributes accumulate, so a later taught line cannot erase an earlier finding. A proposal never presents a disabled service as enabled. A taught line counts as evidence but never makes a foreign file native, so teaching a few lines cannot switch a device back to being scored against defaults. Training progress includes previously taught lines, so it survives a re-audit.

5. Determinism, Honesty and Data Handling

The compliance score is $100 \times (1 - \text{failed_weight} / \text{total_weight})$, severity-weighted, fully deterministic and computed per framework (CIS by default): the same configuration produces the same score on every run.

Evidence gate. A missing line counts as a failure only for files the parser natively reads, meaning at least 60% of code lines recognised; across every tracked configuration, real configs measure 76% or more and probe files 43% or less. For any other

file the score is withheld and the result is reported as **Partial (N of 32 controls)** or **Not assessed**, with only line-backed attributes evaluated. A rule whose resource type was never parsed is never counted as a pass.

AI and network use. An LLM is used only to write attack-path narratives; scoring and detection never depend on model output. Offline mode serves narratives from a cache and deterministic templates with no network calls. In hosted mode a whitelist filter passes only rule and chain metadata to the model; raw configuration lines, remediation text and secrets never leave the machine, and tests enforce this. CVE context is read from a committed offline NVD cache (Cisco IOS only), so the audit path makes no network calls. Text from uploaded configurations is treated as untrusted in every export.

6. Current Implementation Status

- **Vendors:** Cisco IOS, Juniper JunOS and Terraform/AWS natively; other vendors through the training loop (demonstrated on MikroTik and Aruba).
- **Rules:** 54 active. Cisco: 32 cite the CIS Cisco IOS 15 Benchmark v4.1.0 and 4 map to NIST 800-53 (AC-2, AC-3, AU-2, SC-8). AWS: 18 cite the CIS AWS Foundations Benchmark v1.5.0. Three Cisco and four AWS checks are standard hardening outside the benchmark and are labelled as such. Two further AWS rules (AWS-S3-001, AWS-LOG-003) are parked until the Terraform parser emits the attributes they check.
- **Attack chains:** 6 definitions (3 network, 3 cloud); an audit shows only chains whose contributing rules fired. Each network chain has a positive and a negative fixture: applying the chain's own break-chain fix, and nothing else, must stop it.
- **Outputs:** pass/fail per control, severity-weighted score, the config line behind each finding, remediation (labelled as Cisco IOS reference syntax for non-Cisco devices), CVE context for Cisco IOS, PDF and DOCX export, and bulk audit of mixed vendors (POST /api/audit/batch).
- **Frameworks:** CIS implemented, NIST partially implemented. DISA STIG and ISO/IEC 27001 are supported by the architecture (rules carry a framework field; GET /api/frameworks reports coverage) but are not implemented.
- **Quality:** 147 automated tests. Device fields such as serial number and model are reported as "Not available in supplied configuration" when absent, never guessed.

Validation. On a self-generated corpus of 37 configurations, all 74 seeded misconfigurations are detected, with 18 false positives and none on the clean baseline; this is a regression check, not an accuracy claim. On real-world configurations the rules were not tuned against, 31 issues in five configs were hand-labelled in advance, each naming the rule that should catch it and the config line, and the labels were committed before scoring. Detection is **27 of 31 (87.1%)**, all 27 at the labelled line. The misses are RSA key size and proxy-ARP (no rule yet) and two configs relying on IOS's default 10-minute VTY timeout, which the engine treats as compliant. On a sixth, hardened configuration both findings raised were correct. The remaining rule hits on the labelled configs have not been manually reviewed, so no false-positive rate is claimed for them.

7. Where Is the AI?

Cross-vendor normalisation into a common schema; declarative attack-chain correlation that matches findings against authored chain definitions and surfaces the fix that breaks each chain, rendered as a directed graph; concept-matching proposals combined with prefix-based learning, which extend parser coverage without retraining or redeployment; and an LLM used solely for attack-path narrative, with a deterministic template fallback. There is no trained ML model, and compliance scoring never depends on model output: **AI proposes, an administrator decides, deterministic rules audit.**

This is a deliberate boundary, not a gap. No labelled corpus of misconfigured device configurations exists to train a classifier on. A finding must cite a specific control and a configuration line to be auditable in front of a reviewer, which a black-box model cannot provide, and compliance scoring must be reproducible across runs. For the same reason the fix that breaks an attack chain is not computed: each chain declares its own `break_chain.fix_rule` in YAML, authored by a human. The correlator checks that the named fix is among the rules that fired and drops the chain otherwise, and the test fixtures confirm that applying that fix breaks the chain.

8. Scope and Limitations

- **Out of scope by design:** live device scanning (the problem statement asks for configuration-file ingestion, which also suits air-gapped environments) and sandbox exploitation of attack paths, which would be exploitation tooling, needs vendor images that cannot be distributed, and cannot be reproduced by an evaluator.
- **A taught line sets one attribute,** so rules that also depend on a second setting (such as a plaintext password's privilege level) are reported as not evaluated rather than guessed. Remediation is written in Cisco IOS syntax and labelled as reference syntax for other vendors. CIS AWS Foundations v1.5.0 has been superseded by v5.0.0; moving to it is a relabelling of existing rules.